



基于RISC-V代理内核的操作 系统课程实验与课程设计

第五章 . 实验3: 进程管理

目录

- 实验3的基础知识
 - 多任务环境下进程的封装
 - 进程的换入与换出
 - 就绪进程的管理与调度
- 实验内容
 - lab3_1 进程创建 (fork)
 - lab3_2 进程yield
 - lab3_3 循环轮转调度

实验3的基础知识

多任务环境下进程的封装

实验三的PKE操作系统将需要支持多个进程的执行。

操作系统定义了进程池（`process procs[NPROC];`）。并对进程的结构进行了扩充，加入以下成员：

1. `mapped_info;` //points to a page that contains mapped_regions
2. `total_mapped_region;` //next free mapped region in mapped_info
3. `uint64 pid;` //process id,
4. `int status;` //process status,
5. `struct process *parent;` //parent process,
6. `struct process *queue_next;` //next queue element;

多任务环境下进程的封装

进程可能拥有的段分为以下几个类型：进程具有以下几种状态：

```
enum segment_type {
```

- CODE_SEGMENT, // ELF segment
 - DATA_SEGMENT, // ELF segment
 - STACK_SEGMENT, // runtime segment
 - CONTEXT_SEGMENT, // trapframe segment
 - SYSTEM_SEGMENT, // system segment
- ```
};
```

```
enum proc_status {
```

- FREE, // unused state
  - READY, // ready state
  - RUNNING, // currently running
  - BLOCKED, // waiting for something
  - ZOMBIE, // terminated but not reclaimed yet
- ```
};
```

进程的启动

进程启动的步骤：

一、调用`alloc_process()`函数：

1. 找到一个空的进程结构体
2. 为新创建的进程建立了`KERN_BASE`以上逻辑地址的映射
3. 将映射信息保存到进程结构中

二、调用`load_bincode_from_host_elf()`函数载入给定应用对应的ELF文件的各个段

三、调用的`elf_load()`函数在载入段后，将对被载入的段进行判断，以记录它们的虚地址映射，此时将形成用户进程的虚地址空间结构。

四、通过`switch_to()`函数将所构造的进程投入执行；

进程的终止

进程的终止通过exit系统调用实现：

一、调用`free_process( current )`：将进程设为ZOMBIE状态，而不会将进程所占用的资源全部释放。

该函数被调用时操作系统是在S模式下运行，而按照PKE的设计思想，S态的运行将使用当前进程的用户系统栈。此时，如果将当前进程的内存空间进行释放，将导致操作系统本身的崩溃。

二、调用`schedule()`函数：选择系统中可能存在的其他处于就绪状态的进程投入运行。

就绪进程的管理与调度

PKE的操作系统设计了一个非常简单的就绪队列管理。

将一个进程加入就绪队列，可以调用insert_to_ready_queue()函数：

1. 若就绪队列为空，将本进程作为就绪队列头。
2. 若就绪队列非空，则将本进程加入就绪队尾。

PKE内核通过调用schedule()函数来完成进程的选择和换入：

1. 若就绪队列空，检查是否存在未结束的进程。如果有，等待期执行结束；如果无，则启动关机程序。
2. 若就绪队列非空，将就绪队列队首的进程换入执行。

目录

- 实验3的基础知识
 - 多任务环境下进程的封装
 - 进程的换入与换出
 - 就绪进程的管理与调度
- 实验内容
 - lab3_1 进程创建 (fork)
 - lab3_2 进程yield
 - lab3_3 循环轮转调度

lab3_1 进程创建 (fork)

给定应用

- user/app_naive_fork.c

```
1 /*
2  * The application of lab3_1.
3  * it simply forks a child process.
4  */
5
6 #include "user/user_lib.h"
7 #include "util/types.h"
8
9 int main(void) {
10     uint64 pid = fork();
11     if (pid == 0) {
12         printu("Child: Hello world!\n");
13     } else {
14         printu("Parent: Hello world! child id %ld\n", pid);
15     }
16
17     exit(0);
18 }
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_naive_fork
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080010000, PKE kernel size: 0x000000000010000
free physical memory address: [0x0000000080010000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
Switching to user mode...
in alloc_proc. user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
User application is loading.
Application: ./obj/app_naive_fork
CODE_SEGMENT added at mapped info offset:3
Application program entry point (virtual address): 0x0000000000010078
going to insert process 0 to ready queue.
going to schedule process 0 to run.
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087faf000, user stack 0x000000007ffff000, user kstack 0x0000000087fae000
do_fork map code segment at pa:0000000087fb2000 of parent to child at va:0000000000010000.
going to insert process 1 to ready queue.
Parent: Hello world! child id 1
User exit with code:0.
going to schedule process 1 to run.
Child: Hello world!
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

预期输出

lab3_1 进程创建 (fork)

实验内容:

主进程调用fork()函数试图创建一个子进程，但是应用程序的fork动作并未将子进程给创建出来并投入运行。

修改kernel/process.c文件中的do_fork()函数。对于父进程的代码段，通过映射的办法，将子进程中对应的逻辑地址空间映射到其父进程中装载代码段的物理页面。

lab3_2 进程yield

- user/app_yield.c

```
1 /*
2  * The application of lab3_2.
3  * parent and child processes intermittently give up their processors.
4  */
5
6 #include "user/user_lib.h"
7 #include "util/types.h"
8
9 int main(void) {
10     uint64 pid = fork();
11     uint64 rounds = 0xffff;
12     if (pid == 0) {
13         printu("Child: Hello world! \n");
14         for (uint64 i = 0; i < rounds; ++i) {
15             if (i % 10000 == 0) {
16                 printu("Child running %ld \n", i);
17                 yield();
18             }
19         }
20     } else {
21         printu("Parent: Hello world! \n");
22         for (uint64 i = 0; i < rounds; ++i) {
23             if (i % 10000 == 0) {
24                 printu("Parent running %ld \n", i);
25                 yield();
26             }
27         }
28     }
29
30     exit(0);
31     return 0;
32 }
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_yield
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080010000, PKE kernel size: 0x0000000000010000 .
free physical memory address: [0x0000000080010000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
Switching to user mode...
in alloc_proc. user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
User application is loading.
Application: ./obj/app_yield
CODE_SEGMENT added at mapped info offset:3
Application program entry point (virtual address): 0x000000000001017c
going to insert process 0 to ready queue.
going to schedule process 0 to run.
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087faf000, user stack 0x000000007ffff000, user kstack 0x0000000087fae000
do_fork map code segment at pa:0000000087fb2000 of parent to child at va:0000000000010000.
going to insert process 1 to ready queue.
Parent: Hello world!
Parent running 0
going to insert process 0 to ready queue.
going to schedule process 1 to run.
Child: Hello world!
Child running 0
going to insert process 1 to ready queue.
going to schedule process 0 to run.
Parent running 10000
going to insert process 0 to ready queue.
going to schedule process 1 to run.
Child running 10000
going to insert process 1 to ready queue.
```

预期输出

lab3_2 进程yield

实验内容：

yield()函数功能未完善导致使得进程无法让出执行权。

完善yield()函数实现进程执行过程中的主动释放CPU的动作：

- 将当前进程置为就绪状态（READY）；
- 将当前进程加入到就绪队列的队尾；
- 转进程调度。

lab3_3 循环轮转调度

给定应用

```
1 /*
2  * The application of lab3_3.
3  * parent and child processes never give up their processor during execution.
4  */
5
6 #include "user/user_lib.h"
7 #include "util/types.h"
8
9 int main(void) {
10     uint64 pid = fork();
11     uint64 rounds = 100000000;
12     uint64 interval = 10000000;
13     uint64 a = 0;
14     if (pid == 0) {
15         printu("Child: Hello world! \n");
16         for (uint64 i = 0; i < rounds; ++i) {
17             if (i % interval == 0) printu("Child running %ld \n", i);
18         }
19     } else {
20         printu("Parent: Hello world! \n");
21         for (uint64 i = 0; i < rounds; ++i) {
22             if (i % interval == 0) printu("Parent running %ld \n", i);
23         }
24     }
25
26     exit(0);
27     return 0;
28 }
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_two_long_loops
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x000000080000000, PKE kernel end: 0x000000080010000, PKE kernel size: 0x000000000010000
free physical memory address: [0x000000080010000, 0x000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x000000080000000
physical address of _etext is: 0x000000080005000
kernel page table is on
Switching to user mode...
in alloc_proc. user frame 0x000000087fbc000, user stack 0x00000007ffff000, user kstack 0x000000087fbb000
User application is loading.
Application: ./obj/app_two_long_loops
CODE_SEGMENT added at mapped info offset:3
Application program entry point (virtual address): 0x00000000001017c
going to insert process 0 to ready queue.
going to schedule process 0 to run.
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x000000087faf000, user stack 0x00000007ffff000, user kstack 0x000000087fae000
do_fork map code segment at pa:000000087fb2000 of parent to child at va:000000000010000.
going to insert process 1 to ready queue.
Parent: Hello world!
Parent running 0
Parent running 10000000
Ticks 0
Parent running 20000000
Ticks 1
going to insert process 0 to ready queue.
going to schedule process 1 to run.
Child: Hello world!
Child running 0
Child running 10000000
Ticks 2
Child running 20000000
Ticks 3
going to insert process 1 to ready queue.
going to schedule process 0 to run.
```

预期输出

lab3_3 循环轮转调度

实验内容：

两个进程在执行各自循环体时，都没有主动释放CPU的动作。这样的设计会导致某个进程长期占据CPU，而另一个进程无法得到执行。

通过利用时钟中断来实现进程的循环轮转调度，避免由于一个进程的执行体过长，导致系统中其他进程无法得到调度的问题！